

Operating Systems (67808)

Exercise 1: User-Level Threads

Due: ~~7/5/2026~~ 14/5/2026

Note: This exercise is complex.

Start early, and write code you'll be happy to debug (write your code with making the debugging process simple in mind, you'll have less miserable time that way).

Coding Assignment

Introduction

In this assignment, you'll create a static library that creates and manages user-level threads. A potential user will be able to include your library and use it according to its public interface: the *uthreads.h* header file.

Your task is to implement all the functions that make up the uthreads API according to their documentation. You will probably find it necessary to implement internal functions and data structures. These should not be visible outside the library, as they are the private part of your implementation. You are not restricted in their number, signatures, or content. You are required to write clear, readable and efficient code.

Remark: Read the exercise description, the code examples and the man pages thoroughly; they will help clarify things.

Remark: The "clear and readable" code requirement is for you, not for us.

The Threads

Initially, a program consists of the default main thread, whose ID is 0. All other threads will be explicitly created. Each existing thread has a unique thread ID, which is a non-negative integer. The ID given to a new thread must be the smallest non-negative integer not already taken by an existing thread (i.e. if a thread with ID 1 is terminated and then a new thread is spawned, it should receive 1 as its ID). The maximal number of threads the library should support (including the main thread) is `MAX_THREAD_NUM`.

States

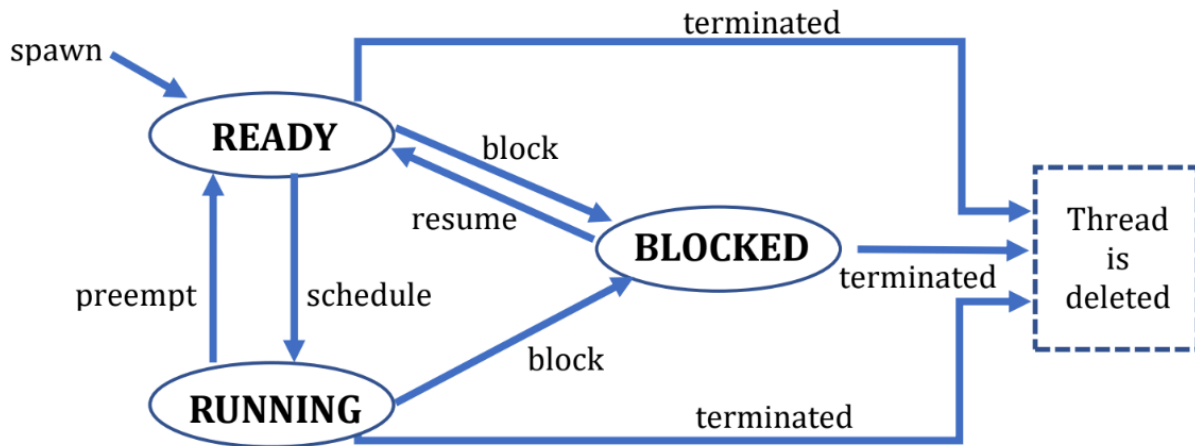
Each thread can be in one of the following states: `RUNNING`, `BLOCKED`, or `READY`:

- `RUNNING` - The thread is currently executing its code.
- `READY` - The thread is ready to run, and waiting for other threads that are currently running.

- **BLOCKED** - The thread cannot be run until another thread resumes it.

Thread State Diagram

At any given time during the running of the user's program, each thread in the program is in one of the states shown in the following state diagram. Transitions from state to state occur as a result of calling one of the library functions, or from elapsing of time, as explained below. This state diagram must not be changed: do not add or remove states.



Scheduler

In order to manage the threads in your library, you will need some sort of scheduling. Your scheduler will implement the Round-Robin (RR) scheduling algorithm.

Time

The process running time is measured by the Virtual Timer.

An example of using this timer can be found in the additional resources in the file *demo_itimer.c*. Note: unless stated otherwise, whenever we mention time in this exercise, we mean the running time of the process (also called the virtual time) and not the real time that has passed in the system.

Algorithm

The Round-Robin scheduling policy should work as follows:

1. Whenever a thread is moved to the **RUNNING** state, it is allocated a predefined number of microseconds to run. This time interval is called a *quantum*.
2. The **RUNNING** thread is preempted if any of the following occurs:
 - a. Its quantum expires.
 - b. It changed its state to **BLOCKED** by calling

- `uthread_block(uthread_get_tid())`
 - c. It entered sleeping (by calling `uthread_sleep()`).
 - d. It is terminated itself using `uthread_terminate(uthread_get_tid())`
- 3. When the RUNNING thread is preempted, do the following:
 - a. If it was preempted because its quantum has expired, move it to the end of the READY threads list.
 - b. Move the next thread in the READY threads list to the RUNNING state.
 - c. Update quantum counters as needed.
- 4. Every time a thread moves to the READY state from any other state, it is placed at the end of the list of READY threads.
- 5. When a thread blocks itself or calls `sleep()`, the scheduler does not preserve its remaining quota. Scheduling continues as if it used all of its allocated time.

Library functions

The file *uthread.h* contains the API of the library. Calling these functions may result in a transition of states in the state diagram shown above. A thread may call a library function with its own ID, thereby possibly changing its own state, or it may call a library function with some other threads ID, thereby affecting the other thread's state.

Notes

1. You are required to manage a queue, list, or a similar data structure of the READY threads. You can use more data structures for other purposes.
2. When a thread gets both blocked and is sleeping, it needs to wait for both the sleeping period to expire and another thread to resume it before going back to the READY threads list.
3. When a thread completes its sleeping time, it is added to the end of the scheduling queue **after** the next scheduling decision is made.
4. The main thread (`tid == 0`) uses the same stack, PC and registers that were used when *uthread_init* is called. There's no need to allocate a stack for it, or to set its SP and PC manually.
Note that this doesn't mean it should behave differently when switching to / from this thread, but just that there is no need to worry about this aspect of its creation.
5. Valgrind doesn't work well with *sigsetjmp* and *siglongjmp*. That's OK. You are not required to configure Valgrind to solve these issues.
You are required to implement your code correctly, and with no memory issues / leaks.
6. Mac owners: The *sigsetjmp* code might not work well on Mac. Unfortunately you'll have to use a virtual machine or SSH to a lab Linux computer.
7. Compilation with -O3 might cause issues, we recommend not using this option.

Simplifying Assumptions

You are allowed to assume the following:

1. All threads end with **uthread_terminate** before returning, either by terminating themselves or due to a call by some other thread.
2. The stack space of each spawned thread isn't exceeded during its execution.
3. The main thread and the threads spawned using the *uthreads* library will not send timer signals themselves (specifically SIGVTALRM), mask them or set interval timers that do so.

Error Messages

The following error messages should be emitted to *stderr*.

Nothing else should be emitted to *stderr* or *stdout*.

When a system call fails (such as memory allocation) you should print a **single line** in the following format:

"system error: *text*\n"

Where *text* is a description of the error, and then *exit(1)*.

When a function in the threads library fails (such as invalid input), you should print a **single line** in the following format:

"thread library error: *text*\n"

Where *text* is a description of the error, and then return the appropriate return value.

Background reading and Resources

1. Read the following man-pages for a complete explanation of relevant system calls:
 - *setitimer* (2)
 - *getitimer* (2)
 - *sigaction* (2)
 - *sigsetjmp* (3)
 - *siglongjmp* (3)
 - *sigprocmask* (2)
 - *sigemptyset*, *sigaddset*, *sigdelset*, *sigfillset*, *sigismember* (3)
 - *sigpending* (2)
 - *sigwait* (3)
2. These examples may help you in your coding:
 - **demo_singInt_handler.c** (ex2 resources) which contains an example of using signal actions.
 - **demo_itimer.c** (ex2 resources) contains an example of using the virtual timer.
 - **demo_jmp.c** (ex2 resources) contains an example of using *sigsetjmp* and *siglongjmp* as demonstrated in class. Note that you must use *translate_address* in your code as seen in the demo, otherwise your code will not work correctly.
3. This code is complex and may be hard to follow; it may be helpful to use the GNU Debugger (**GDB**) to step through your code, set breakpoints, and inspect variables during development.

Running school solution

The school solution can be run as follows: `~os/ex/ex1/run_school_solution <path to a source file 1> [<path to a source file 2> ...]`

The command will compile the school solution with the files you include, run the solution, and display the output. For example, if you wrote a test that contains a `main` function and it is located in the file `/path/to/test.cpp` (in the current directory), you should run:

```
~os/ex1/run_school_solution /path/to/test.cpp
```

You can add multiple files for compilation, and you can also pass a directory (in which case, all files in the directory and its subdirectories will be compiled).

You must check against the school solution before asking questions in the forum—in the event of any conflicts or discrepancies regarding the exercise solution, the school solution dictates the correct approach. To keep the forum clean, we will delete questions (without prior notice) if their answers can be verified using the school solution.

Step by step instructions

This section is here to help decompose the task into manageable steps. It is highly recommended to follow these steps in the implementation.

0. Let's go

Download all additional files into a Linux machine. Use your Linux device, a virtual machine, or the University computers remotely or physically. Place them in a single directory and open it with your favorite IDE. We recommend VS Code with the VS Code Remote plugin to work directly on the Lab computers.

Make sure `gcc/g++` is properly installed by running:

```
g++ --version
```

Then compile the kick-the-tires test:

```
g++ -Wall -Werror -Wpedantic --std=c++17 -I. *.cpp \
    tests/test_1_kick_tires.cpp -o program
./program
```

This should compile, and print the following:

```
thread library error: did not implement
uthread_init failed with code -1
```

Now run it via the Python runner:

```
/usr/bin/python3 tests/run_tests.py test_1_kick_tires
```

You should see something similar to:

```
Test: test_1_kick_tires
```

```
→ Compiling...
```

```
✓ Compilation successful
```

```
→ Running...
```

```
✗ Execution failed
```

```
Exit code: 255
```

```
Stderr: thread library error: did not implement
```

Now run it with the school solution: `~os/ex/ex1/run_school_solution \`
`tests/test_1_kick_tires.cpp`

You should see the output:

```
My thread ID: 0
```

Finally, run it using a debugger. Take the time to make sure this is set up properly. You may consult friends, ChatGPT, Stack Overflow or any resource you need for that step, which is technical and will save you a lot of time in the future.

Make sure you can put a breakpoint, run in debug mode, and print the values of variables.

1. One single thread of gold

Implement the functions `uthread_init` and `uthread_get_tid` in order to make the test pass. Since we only have one thread which is the main thread (id=0), the implementation should be trivial.

- Run the test manually, and from the Python script. Make sure both passes.

2. Forever trusting who we are

Implement the `uthread_spawn` and `uthread_terminate` API functions for thread ID allocation and thread termination.

- It is recommended to have a dedicated class for managing the thread ID allocation. This way that complexity is encapsulated and the code is cleaner and easier to manage and debug.
- For now, all created threads will be in READY state indefinitely until terminated. Do not implement a context switch just yet.

And test it:

- Make sure the test `test_2_thread_id_management` passes.
- Write more tests covering edge cases. For example, terminating a thread that does not exist or spawning and deleting a thread repeatedly to detect memory leaks.

3. Sleep with one eye open

Implement `uthread_sleep` for `num_quantums==0` (return -1 otherwise for now) and the `uthread_get_total_quantums` and `uthread_get_quantums` functions. The effect of zero sleep is giving up the current execution right, voluntarily getting to the end of the queue and letting other threads run.

- Use `demo_jmp.c` as an example on how to perform a context switch once the main thread sleeps.
- The logic of the context switch should be in a separate function, since we will call it from other functions later.
- Count the number of quantum each context switch.

And test it:

- Make sure `test_3_self_preemption` passes.
- Make sure all previous tests still pass.

3.5. Such a breakable thread

Write a test with one spawned thread calling `terminate()` on another spawned thread. Make sure it passes.

- Make sure `uthread_get_quantums` and `uthread_get_total_quantums` count quanta correctly for self-terminating and externally-terminating threads.

4. Jenny from the block (and resume)

The next thing to implement is letting threads block and resume each other.

Implement the functions `uthread_block` and `uthread_resume`.

- Blocked threads do not run until resumed by another thread.
- Resumed threads are added to the end of the queue.
- A thread can block itself (but not resume itself, obviously).

Testing:

- Write a test for the block/resume behaviour. You can validate the test via running the school solution, see the “Running school solution” section.
- Blocked threads introduces some interesting edge cases (for example, thread being terminated while blocked). Make sure to add tests that cover them - Better tackle these bugs right now than after more features are implemented.
- Of course, cover the error handling, like calling `uthread_resume` on a thread that does not exist.

5. Wake me up when September ends

Implement `uthread_sleep` for non-zero quantum.

- Sleeping threads are blocked until their sleep quota is over.
- When the time comes, they may become active again - if no other thread called `uthread_block` on them in the meantime.

Testing

- Make a test for the new code.

- Test for edge cases (for example, multiple threads waking up at the same time)
 - Specifically, test the case of a thread sleeping, then another thread calls `uthread_block` on it - it should not run until its sleep quota is over **and** `uthread_resume` is called.

6. TiK ToK

Implement a timer that will preempt processes after the given microseconds. Preempted threads will move to the end of the queue, while the previously waiting threads executes.

- Use the `demo_itimer.c` as inspiration.

Testing

- Make a test and make sure it passes.
- Add tests to cover different aspects and edge cases.

7. (Optional). Every game you play

At that step your code should be ready for submission. Let's see what it can do!

Under the folder "games" you will find 2 short, fun programs I created with some help from Gemini. Compile them with your library and see how they run.

Submission

Submit a tar or a zip file that contains:

1. README file built according to the course guidelines.
2. All relevant source code files (except `uthreads.h`).

Make sure that the tar file can be extracted and that the extracted files compile.

Guidelines

1. **Read the course guidelines.**
2. Design your program carefully before you start writing it. Pay special attention to choosing suitable data structures.
3. Do not forget to take care of possible signal races - protect relevant code by blocking and unblocking signals at the right places.
4. Encapsulate important actions such as performing a thread switch and deciding which thread should run next. Make sure each action works on its own before combining them.
5. Always check the return value of the system calls you use.
6. Test your code thoroughly - write test programs and cross test programs with other groups.
7. During development, use asserts and debug printouts, but make sure to remove all asserts and any debug output from the library before submission.

Good luck!



MULTITHREADING

THREADS ARE NOT GOING TO SYNCHRONIZE THEMSELVES